

Final Project

Transformer-Based Neural Machine Translation

Natural Language Processing

AIG230NAA

SENECA'S ACADEMIC INTEGRITY POLICY

As a Seneca student, you must conduct yourself in an honest and trustworthy manner in all aspects of your academic career. A dishonest attempt to obtain an academic advantage is considered an offense and will not be tolerated by the Polytechnic.

In this project, you will design and build a complete Neural Machine Translation (NMT) system using a Transformer architecture. The objective is to develop a model capable of translating sentences from one language to another by learning from bilingual sentence pairs. This project is intended to simulate a real-world NLP development pipeline, requiring you to move beyond simply training a model and instead build an end-to-end machine learning solution.

It is recommended to create a virtual environment (using `virtualenv`) specifically for this project if you are planning to run it locally.

The project can be completed in groups of **up to 2 individuals**.

You are expected to complete every stage of the project, beginning with data collection and exploration, followed by preprocessing, model implementation, training, evaluation, error analysis, and finally the development of an interactive inference application. Your final submission should demonstrate not only the performance of your model, but also your understanding of the decisions made throughout the development process.

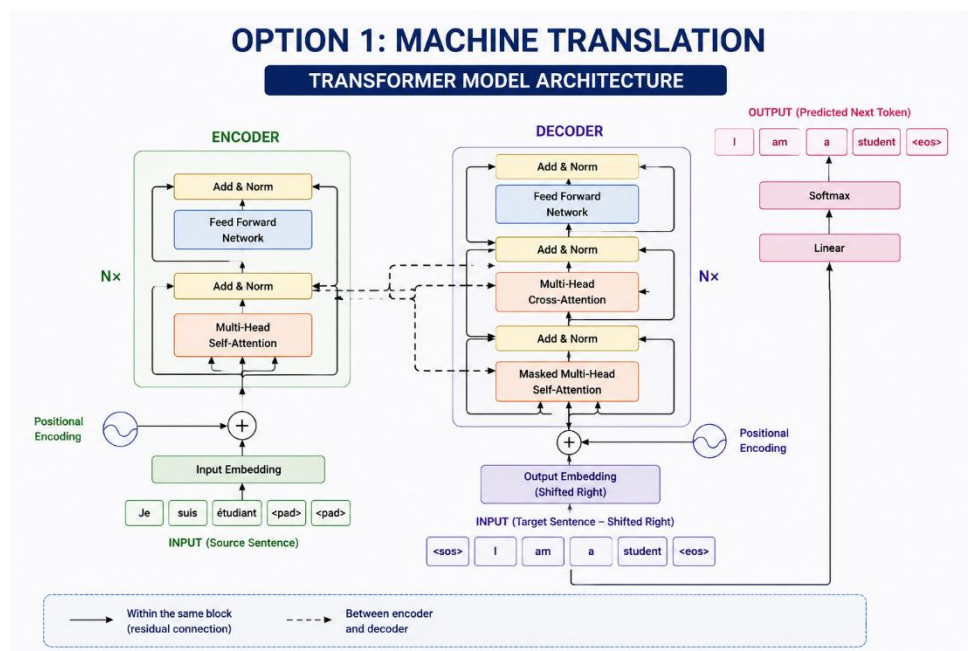
For your dataset, you may choose the [Tatoeba Project](https://tatoeba.org/), and select a language pair of your choice (for example, English–Spanish, English–French, or English–German).



Your first task is to carefully explore the dataset before building the model. Provide a brief analysis describing the size of the dataset, the number of sentence pairs, vocabulary statistics, average sentence lengths, and any characteristics that may influence the translation task. Supporting your observations with appropriate visualizations is highly encouraged.

Next, prepare the data for training by implementing a complete preprocessing pipeline. This includes cleaning the text, tokenizing the sentences, constructing the vocabulary (or using an appropriate subword tokenizer), creating attention masks, padding sequences where necessary, and splitting the dataset into training, validation, and testing sets. Every preprocessing decision should be clearly justified.

You will then implement a Transformer-based translation model using TensorFlow or PyTorch. The architecture should include the essential Transformer components discussed throughout the course, including positional encoding, multi-head attention, encoder and decoder blocks, feed-forward networks, residual connections, and layer normalization. While you may use built-in deep learning layers provided by the framework, the overall model architecture and training pipeline must be implemented by you rather than relying on high-level automated training utilities.



Once the model has been implemented, train it using an appropriate optimization strategy and carefully monitor the learning process. You should present and discuss the training and validation curves, explain your choice of hyperparameters, and describe any techniques used to improve model performance.

After training, evaluate your translation system using the BLEU score.

An important component of this project is error analysis. Rather than reporting only the evaluation metrics, examine at least twenty translation examples where the model performs poorly. Identify common failure cases such as incorrect word order, repeated words, missing information, unknown vocabulary, or difficulties with long sentences, and discuss possible reasons for these errors as well as potential improvements.

Finally, you are required to develop a complete inference application that allows a user to interact with your trained model. The application should load the saved model, accept any input sentence from the user, generate a translation without retraining the model, and display the predicted output in real time.

The final submission should include your complete source code, trained model weights, a well-organized report documenting your methodology and experimental results, and a demonstration of your inference application. It is highly recommended to record a short video of the final result at inference time. Your project will be evaluated not only on model performance but also on the quality of your implementation, analysis, and ability to build a complete, production-style NLP system.

Deliverables

At the end of the project, you are expected to submit a complete package that demonstrates every stage of your work, from model development to deployment. The following items are required:

1. Source Code: Submit all source code used throughout the project.
2. Trained Model: Include the final trained model (weights/checkpoints) used during inference.
3. Project Report: Include a comprehensive README that explains how to install dependencies, set up the environment, train the model (if desired), load the trained model, and run the inference application.
4. Demonstration Video

Evaluation Rubric

Criteria	Weight	Poor	Fair	Good	Very Good	Excellent
Data Collection & Exploration	10%	Incomplete or poor dataset selection and analysis.	Basic exploration.	Good analysis.	Thorough analysis with visualizations.	Comprehensive and insightful analysis.
Data Preprocessing	10%	Incomplete or incorrect.	Basic preprocessing.	Mostly correct.	Well-designed pipeline.	Excellent, efficient preprocessing.
Transformer Model Development	25%	Incomplete or incorrect model.	Partially correct implementation.	Correct with minor issues.	Well-implemented architecture.	Excellent implementation and design.
Training Pipeline	15%	Incomplete or incorrect training.	Basic training pipeline.	Functional with appropriate settings.	Well-designed and monitored.	Robust, optimized training pipeline.
Evaluation & Baseline Comparison	15%	Incomplete evaluation.	Basic evaluation.	Appropriate metrics and comparison.	Strong evaluation and discussion.	Comprehensive and insightful evaluation.
Error Analysis	10%	Little or no analysis.	Basic analysis.	Reasonable insights.	Thorough analysis.	Deep and insightful analysis.
Inference Application	10%	Incomplete or non-functional.	Basic functionality.	Functional application.	User-friendly application.	Professional-quality application.
Code Quality	5%	Poor organization.	Basic organization.	Clean and readable.	Well-structured and documented.	Excellent, modular, professional code.

Resources to help:

- [PEP8 – Style Guide for Python Code](#)
- [Machine Learning Project Structure](#)
- [The illustrated Transformer](#)